

Predictive SDN Dynamic Load Balancer

End-to-End Technical Documentation

Project: Predictive-SDN-Dynamic-Load-Balancer

Author: Samyak Anand

Current Stage: Data Ingestion → Kafka → Validation → Kafka → PostgreSQL

Status: End-to-End data ingestion and validation pipeline successfully tested with 10 records

1. Project Overview

The **Predictive SDN Dynamic Load Balancer** is a data-driven networking system designed to analyze network traffic and eventually use machine learning to predict traffic conditions and dynamically optimize network load distribution.

The project combines:

- Software-Defined Networking (SDN)
- Network traffic datasets
- XML data ingestion
- Apache Kafka
- Data validation
- Dead Letter Queue (DLQ)
- PostgreSQL
- Feature engineering
- Machine Learning
- MLflow
- MLOps
- Docker
- Real-time/streaming data processing

The current implementation focuses on building the foundation of the system:

Historical XML network traffic → Kafka → Validation → Kafka → PostgreSQL

Once this data pipeline is stable, the next stages will consume the validated traffic data for feature engineering, ML model training, prediction, and eventually dynamic load-balancing decisions.

2. Project Objective

The main objective is to develop a predictive load-balancing architecture capable of identifying network traffic conditions before congestion occurs.

Traditional load balancing generally reacts to current network conditions.

This project aims to move toward a predictive approach:

Traditional:

```
Traffic
  ↓
Congestion
  ↓
Detect congestion
  ↓
React
  ↓
Load balancing
```

The proposed architecture aims to become:

```
Traffic
  ↓
Real-time ingestion
  ↓
Data validation
  ↓
Feature engineering
  ↓
ML prediction
  ↓
Predict future traffic/congestion
  ↓
Dynamic load balancing
```

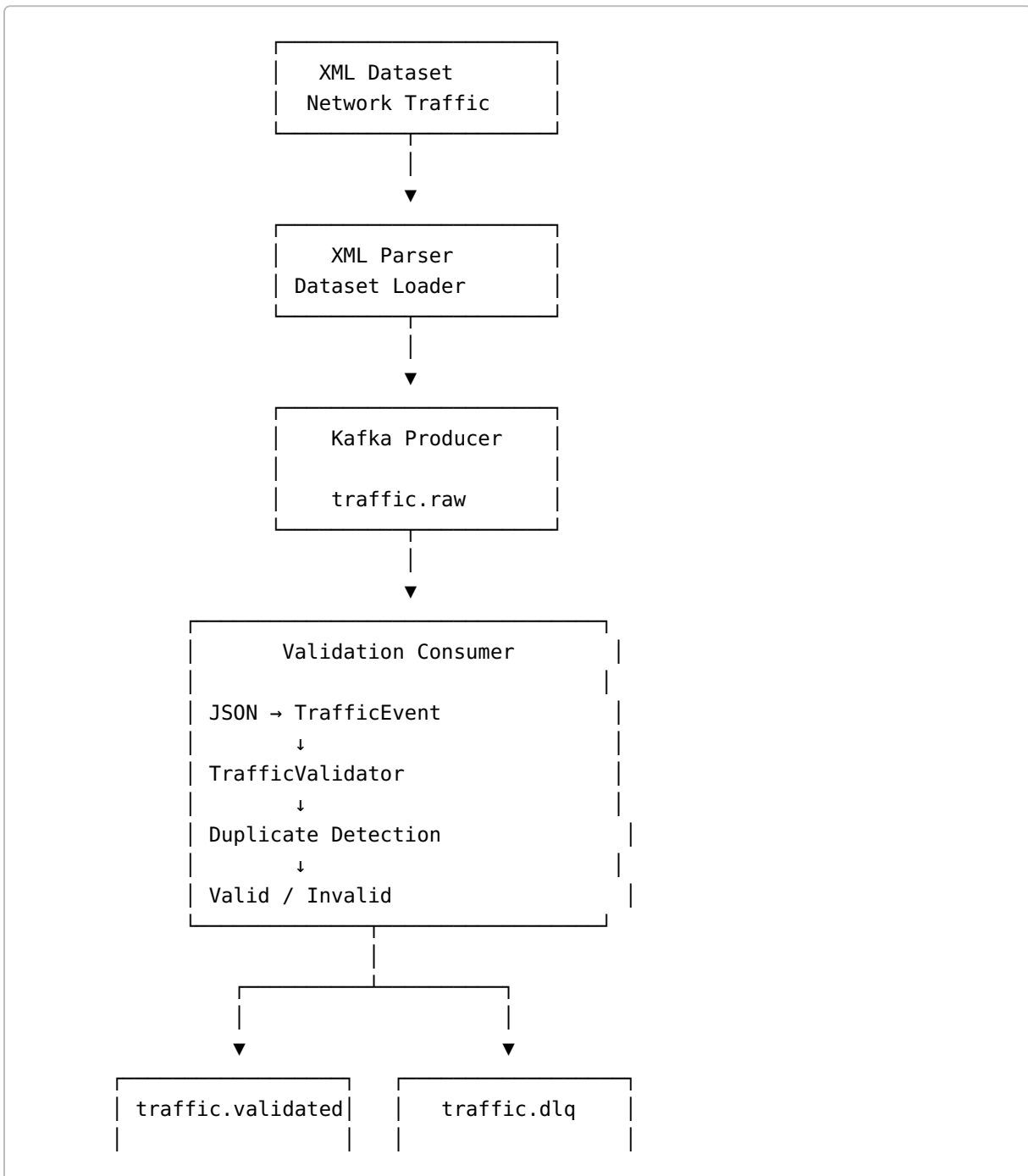
The ML component will eventually learn traffic patterns and predict network conditions such as:

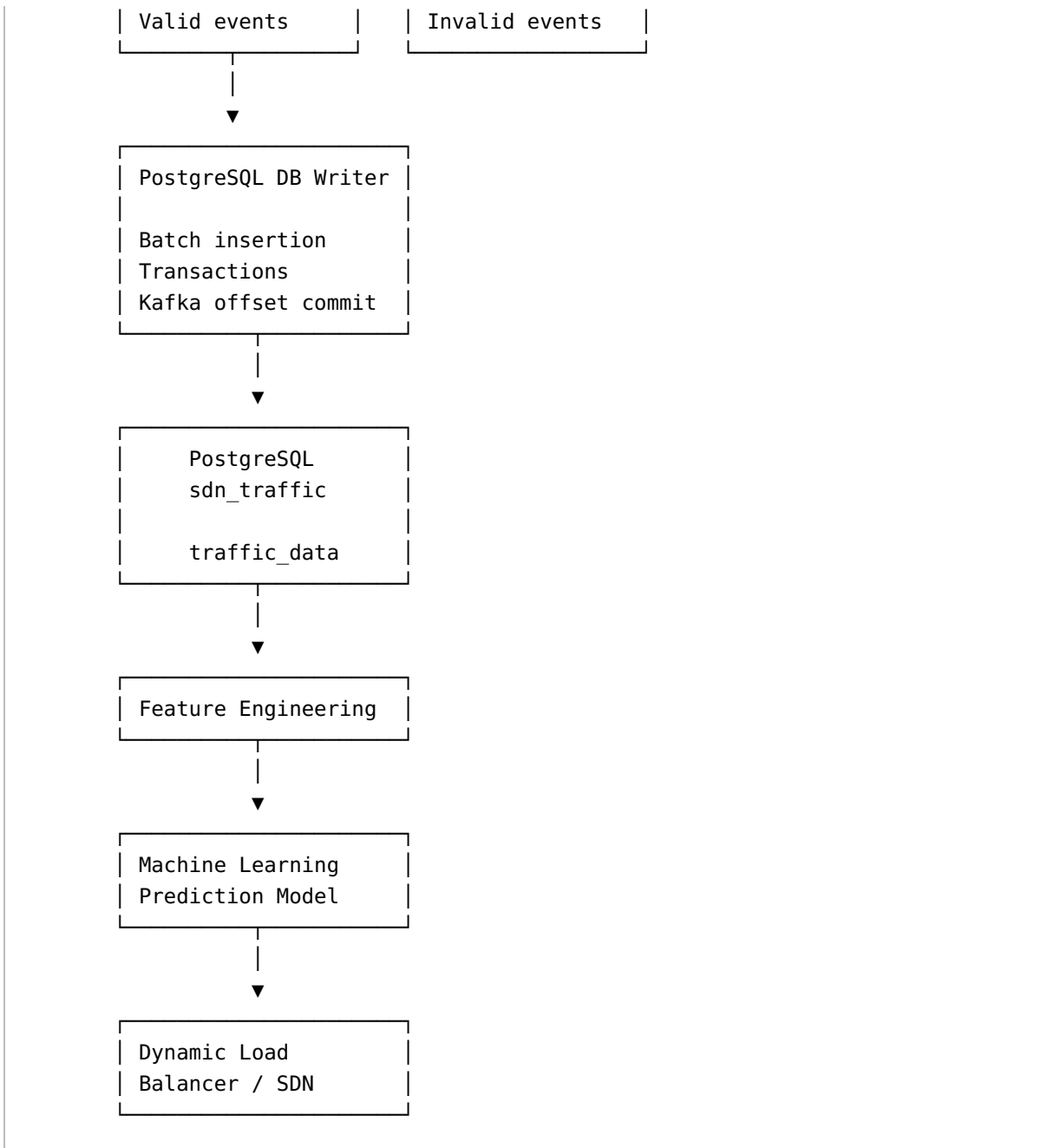
- traffic demand
- throughput
- congestion
- latency
- packet loss

- network utilization
- potentially overloaded links/nodes

3. High-Level Architecture

The current architecture is:





4. Current Implementation Status

Component	Status
Project structure	Completed
Python virtual environment	Completed

Component	Status
Docker environment	Completed
Kafka	Running
PostgreSQL	Running
Kafka <code>traffic.raw</code> topic	Completed
Kafka <code>traffic.validated</code> topic	Completed
XML dataset loading	Completed
XML parsing	Completed
Traffic event model	Completed
Traffic validation	Completed
Duplicate detection	Implemented
Raw Kafka producer	Completed
Kafka raw consumer	Completed
Validation consumer	Completed
DLQ producer	Implemented
PostgreSQL schema	Completed
Kafka → PostgreSQL writer	Completed
10-record integration test	PASSED
Large-scale ingestion	Pending
Feature engineering	Pending
ML model	Pending
MLflow	Pending
Dynamic SDN control	Pending

5. Project Directory Structure

The current project structure is:

```
Predictive-SDN-Dynamic-Load-Balancer/
|
```

```
├── config.yaml
├── data/
├── docker/
│   └── kafka/
│       └── docker-compose.yml
├── README.md
├── scripts/
│   ├── __init__.py
│   └── produce_traffic.py
├── src/
│   ├── __init__.py
│   ├── kafka/
│   │   ├── consumer.py
│   │   ├── dlq_producer.py
│   │   └── producer.py
│   ├── loaders/
│   │   └── dataset_loader.py
│   ├── models/
│   │   └── traffic_event.py
│   ├── parsers/
│   │   ├── sndlib_parser.py
│   │   └── xml_parser.py
│   ├── pipelines/
│   │   ├── validation_pipeline.py
│   │   └── validation_consumer.py
│   ├── validators/
│   │   ├── duplicate_detector.py
│   │   └── traffic_validator.py
│   └── database/
│       ├── __init__.py
│       └── db_writer.py
└── tests/
```

6. Environment Setup

The project uses a Python virtual environment.

The environment is activated with:

```
source .venv/bin/activate
```

After activation the shell displays:

```
(.venv)
```

This is important because the operating system's Python environment is externally managed.

7. Python Dependencies

The project uses Python libraries including:

- kafka-python
- psycopg
- psycopg[binary]
- XML parsing libraries
- Pandas
- NumPy
- Scikit-learn
- PyYAML
- MLflow
- other ML/data-engineering dependencies planned for later stages

The Kafka client was verified successfully:

```
kafka-python OK
```

PostgreSQL connectivity was also verified through:

```
psycopg OK: 3.3.5
```

8. Docker Infrastructure

The infrastructure is managed using Docker Compose.

The compose file is:

```
docker/kafka/docker-compose.yml
```

The environment currently contains:

```
Kafka 4.0.0  
PostgreSQL 16
```

Kafka is exposed on:

```
localhost:9092
```

PostgreSQL is exposed on:

```
localhost:5432
```

9. PostgreSQL Configuration

The PostgreSQL database is:

```
Database: sdn_traffic  
User: sdn_user  
Password: sdn_password  
Host: localhost  
Port: 5432
```

The main table is:

```
traffic_data
```

10. PostgreSQL Schema

The current `traffic_data` table contains:

```
id
event_id
event_timestamp
source_node
destination_node
source_ip
destination_ip
protocol
packet_count
byte_count
duration_ms
throughput_mbps
packet_loss_percent
latency_ms
raw_payload
kafka_topic
kafka_partition
kafka_offset
ingested_at
```

Detailed schema:

Column	Type	Purpose
id	bigint	Primary database identifier
event_id	text	Unique traffic event identifier
event_timestamp	timestampz	Event timestamp
source_node	text	Source network node
destination_node	text	Destination network node
source_ip	inet	Source IP address
destination_ip	inet	Destination IP address
protocol	text	Network protocol
packet_count	bigint	Number of packets
byte_count	bigint	Number of bytes
duration_ms	double precision	Flow duration

Column	Type	Purpose
throughput_mbps	double precision	Traffic throughput
packet_loss_percent	double precision	Packet loss
latency_ms	double precision	Network latency
raw_payload	jsonb	Original Kafka message
kafka_topic	text	Source Kafka topic
kafka_partition	integer	Kafka partition
kafka_offset	bigint	Kafka offset
ingested_at	timestamptz	Database ingestion timestamp

11. Database Constraints

A unique constraint was added on:

```
(kafka_topic, kafka_partition, kafka_offset)
```

Constraint name:

```
uq_kafka_message
```

This is important for idempotency.

Kafka identifies a message using:

```
topic + partition + offset
```

Therefore, if the DB writer receives the same Kafka message again, PostgreSQL can prevent duplicate insertion.

The insertion uses:

```
ON CONFLICT (kafka_topic, kafka_partition, kafka_offset)
DO NOTHING
```

12. Kafka Architecture

Kafka is the streaming backbone of the project.

The current topics are:

```
traffic.raw
traffic.validated
```

A DLQ topic is also supported:

```
traffic.dlq
```

13. traffic.raw

The `traffic.raw` topic is the entry point for traffic events.

Current configuration:

```
Partitions: 3
Replication factor: 1
```

The architecture is:

```
XML
↓
Dataset Loader
↓
Producer
↓
traffic.raw
```

The three partitions allow Kafka to distribute traffic across multiple processing workers.

14. traffic.validated

The `traffic.validated` topic contains events that passed validation.

Architecture:

```
traffic.raw
  ↓
Validation Consumer
  ↓
TrafficValidator
  ↓
traffic.validated
```

It currently also has:

```
Partitions: 3
Replication factor: 1
```

15. Kafka Partitioning

During the test, the 10 messages were distributed across the three raw partitions:

```
traffic.raw:0 → 3 messages
traffic.raw:1 → 2 messages
traffic.raw:2 → 5 messages
```

Therefore:

```
Total = 3 + 2 + 5 = 10
```

This confirms that Kafka partitioning is functioning.

16. XML Dataset

The project uses the Abilene network traffic dataset.

The dataset path used during testing was:

```
/home/samyak/PycharmProjects/Predictive-SDN-Dynamic-Load-Balancer_data/directed-abilene-zhang-5min-over-6months-ALL-native
```

The dataset contains historical network traffic measurements represented through XML files.

The final target is to process the complete dataset, which contains millions of records.

However, before processing millions of records, a small integration test was intentionally performed using only 10 records.

This is an important engineering practice.

Instead of immediately processing millions of records, the pipeline was validated using:

```
10 records
```

This allows failures to be detected before large-scale ingestion.

17. Dataset Loader

The file:

```
src/loaders/dataset_loader.py
```

is responsible for loading the dataset.

The producer uses:

```
DatasetLoader(dataset_name="abilene")
```

and loads the configured dataset.

The loader abstracts the dataset source from the Kafka producer.

Therefore, the producer does not need to know the internal structure of the XML files.

18. XML Parser

The project contains:

```
src/parsers/xml_parser.py
src/parsers/sndlib_parser.py
```

The purpose of the parser layer is to convert XML network traffic information into Python objects.

The general process is:

```
XML file
  ↓
XML parser
  ↓
Raw network traffic information
  ↓
TrafficEvent
```

This separation is important because parsing logic remains independent of Kafka.

19. Traffic Event Model

The normalized event representation is defined in:

```
src/models/traffic_event.py
```

The event contains information such as:

```
event_id
timestamp
source_node
destination_node
traffic_mbps
demand_id
granularity
unit
dataset
source_format
source_folder
```

```
source_file
schema_version
```

This normalized model provides a consistent structure regardless of the original XML representation.

20. Why Normalize the XML Data?

Raw XML structures can be difficult to process directly.

Instead of allowing every downstream component to understand XML, the project converts data into a standardized event:

```
XML
↓
Parser
↓
TrafficEvent
```

Then:

```
TrafficEvent
↓
Kafka
↓
Validation
↓
PostgreSQL
↓
ML
```

This reduces coupling between components.

21. Validation Pipeline

The validation logic is implemented in:

```
src/pipelines/validation_pipeline.py
```

It combines:

```
TrafficValidator
+
DuplicateDetector
```

The pipeline can classify events as:

```
VALID
INVALID
DUPLICATE
```

22. Traffic Validator

The file:

```
src/validators/traffic_validator.py
```

is responsible for checking whether a traffic event satisfies the expected rules.

The validator operates on the normalized `TrafficEvent`.

Conceptually:

```
TrafficEvent
  ↓
Validation Rules
  ↓
Valid / Invalid
```

Invalid messages can be sent to the DLQ.

23. Duplicate Detector

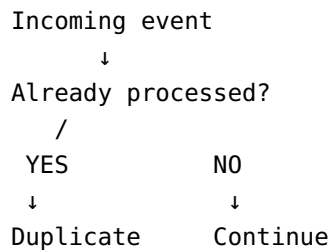
The file:

```
src/validators/duplicate_detector.py
```

is responsible for detecting duplicate events.

Duplicate detection is important because network traffic datasets may contain repeated records and because streaming systems can provide messages more than once.

The overall objective is:



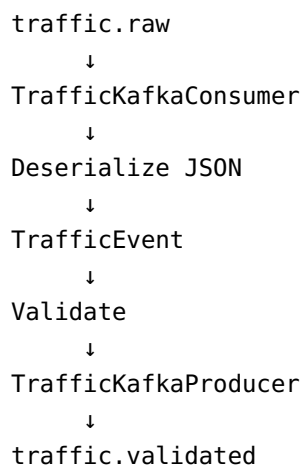
24. Validation Consumer

A dedicated validation bridge was created:

```
src/pipelines/validation_consumer.py
```

Its responsibility is to connect the raw and validated Kafka topics.

The flow is:



This component solved an important architectural gap.

Initially, the existing consumer only yielded validated events; it did not publish them to the `traffic.validated` topic.

The new validation consumer provides the missing bridge.

25. Raw Kafka Consumer

The file:

```
src/kafka/consumer.py
```

contains:

```
TrafficKafkaConsumer
```

It performs:

1. Kafka consumption
2. JSON deserialization
3. `TrafficEvent` reconstruction
4. validation
5. DLQ handling
6. yielding valid events

The consumer uses:

```
topic = traffic.raw  
group_id = traffic-validation-consumer
```

26. Invalid Data Handling

If a message cannot be deserialized:

```
Kafka message  
↓  
JSON decoding  
↓  
ERROR  
↓  
traffic.dlq
```

Similarly, if the event cannot be reconstructed or validation fails:

```
Invalid Event
  ↓
traffic.dlq
```

This prevents malformed records from stopping the entire pipeline.

27. Dead Letter Queue

The DLQ producer is implemented in:

```
src/kafka/dlq_producer.py
```

The DLQ topic is:

```
traffic.dlq
```

The purpose of the DLQ is to preserve invalid messages for later investigation.

A production system should not simply discard malformed records.

Instead:

```
Valid
  ↓
Normal pipeline

Invalid
  ↓
DLQ
  ↓
Investigate / Correct / Replay
```

28. Kafka Producer

The main Kafka producer is:

```
src/kafka/producer.py
```

Class:

```
TrafficKafkaProducer
```

The producer converts a `TrafficEvent` into JSON.

Example structure:

```
{
  "event_id": "...",
  "timestamp": "...",
  "source_node": "...",
  "destination_node": "...",
  "traffic_mbps": 0.0,
  "demand_id": "...",
  "granularity": "...",
  "unit": "...",
  "dataset": "abilene",
  "source_format": "xml",
  "source_folder": "...",
  "source_file": "...",
  "schema_version": "1.0"
}
```

29. Kafka Producer Reliability Configuration

The producer uses:

```
acks = all
retries = 5
linger_ms = 10
batch_size = 32768
compression_type = gzip
```

The producer is also configured as an idempotent producer by the Kafka client.

These settings improve reliability and throughput.

30. Data Injection Script

The main ingestion script is:

```
scripts/produce_traffic.py
```

It performs:

```
DatasetLoader
  ↓
ValidationPipeline
  ↓
TrafficKafkaProducer
  ↓
traffic.raw
```

The script supports:

```
--limit
```

which was extremely useful for testing.

31. First Producer Test

The initial command was:

```
python scripts/produce_traffic.py --limit 10
```

This resulted in:

```
ModuleNotFoundError: No module named 'src'
```

The reason was related to Python module resolution when executing the script directly.

The corrected command was:

```
python -m scripts.produce_traffic --limit 10
```

This successfully resolved the package imports.

32. Successful XML → Kafka Test

The corrected producer command:

```
python -m scripts.produce_traffic --limit 10
```

successfully processed 10 records.

The result was:

```
Input events: 10
Produced events: 10
Failed events: 0
```

The measured throughput was approximately:

```
4.58 events/sec
```

This confirmed that the following stage was working:

```
XML Dataset
  ↓
Dataset Loader
  ↓
Parser
  ↓
TrafficEvent
  ↓
Kafka Producer
  ↓
traffic.raw
```

33. Raw Kafka Result

Kafka offsets were checked using:

```
docker exec -it predictive-sdn-kafka  
/opt/kafka/bin/kafka-get-offsets.sh  
--bootstrap-server localhost:9092  
--topic traffic.raw
```

The result was:

```
traffic.raw:0:3  
traffic.raw:1:2  
traffic.raw:2:5
```

Therefore:

```
Partition 0 = 3  
Partition 1 = 2  
Partition 2 = 5  
  
Total = 10
```

This confirmed that all 10 test records successfully entered Kafka.

34. Validation Consumer Test

The validation consumer was executed with:

```
source .venv/bin/activate  
  
python -m src.pipelines.validation_consumer
```

Kafka successfully connected:

```
Broker version identified as 4.0
```

The consumer successfully joined:

```
traffic-validation-consumer
```

It eventually received all three partitions:

```
traffic.raw partition 0  
traffic.raw partition 1  
traffic.raw partition 2
```

35. Validation Test Result

The validation consumer reported:

```
Processed valid events : 10  
Published validated    : 10
```

This is a major integration test result.

It confirms:

```
traffic.raw  
  ↓  
Kafka Consumer  
  ↓  
JSON Deserialization  
  ↓  
TrafficEvent  
  ↓  
Validation  
  ↓  
traffic.validated
```

All 10 test records successfully passed this stage.

36. Validated Kafka Topic Result

The validation consumer produced the events into:


```
traffic.validated
```

The logs showed successful writes such as:

```
ENTRY WRITTEN | event_id=... | topic=traffic.validated | partition=...
```

The validated messages were distributed across Kafka partitions.

This confirms that the raw-to-validated Kafka bridge is functioning.

37. PostgreSQL DB Writer

The PostgreSQL writer is:

```
src/database/db_writer.py
```

Its responsibility is:

```
Kafka traffic.validated
      ↓
PostgreSQL DB Writer
      ↓
PostgreSQL
```

It consumes validated messages and stores them in:

```
traffic_data
```

38. DB Writer Configuration

The DB writer uses:

```
Kafka:
localhost:9092

Topic:
```

```
traffic.validated

Consumer group:
postgres-db-writer

PostgreSQL:
localhost:5432

Database:
sdn_traffic

User:
sdn_user

Batch size:
5000

Flush interval:
5 seconds
```

39. DB Writer Reliability

The database writer uses:

```
enable_auto_commit = False
```

This is important.

The Kafka offset is not automatically committed before the database transaction completes.

The intended flow is:

```
Kafka message
  ↓
Prepare DB record
  ↓
PostgreSQL transaction
  ↓
COMMIT
  ↓
Kafka offset COMMIT
```

This reduces the possibility of losing data when a failure occurs between Kafka consumption and database insertion.

40. PostgreSQL Transaction

The DB writer processes records in batches.

For example:

```
5000 records
  ↓
PostgreSQL transaction
  ↓
INSERT
  ↓
COMMIT
  ↓
Kafka offset commit
```

For the current test:

```
10 records
```

were processed as a batch.

41. Database Integration Test

The DB writer was started with:

```
source .venv/bin/activate

python src/database/db_writer.py
```

The logs confirmed:

```
Starting Kafka → PostgreSQL DB Writer
```

Kafka connection succeeded:

Broker version 4.0

PostgreSQL connection succeeded:

Connected to PostgreSQL successfully

The consumer received all three partitions:

```
traffic.validated partition 0  
traffic.validated partition 1  
traffic.validated partition 2
```

42. PostgreSQL Insert Result

The DB writer reported:

Inserted batch of 10 records into PostgreSQL

Then:

Kafka offsets committed successfully

This is one of the most important results of the current implementation.

It confirms that:

```
Kafka  
  ↓  
Validated Kafka Topic  
  ↓  
DB Writer  
  ↓  
PostgreSQL
```

is functioning successfully.

43. Final Database Verification

The database was verified using:

```
docker exec -it predictive-sdn-postgres
psql -U sdn_user -d sdn_traffic
-c "SELECT COUNT(*) FROM traffic_data;"
```

The result was:

```
count
-----
      10
(1 row)
```

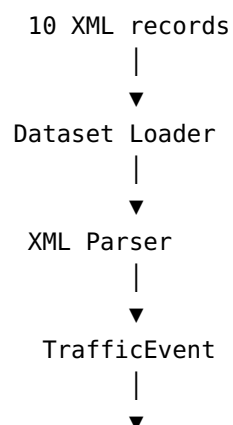
Therefore:

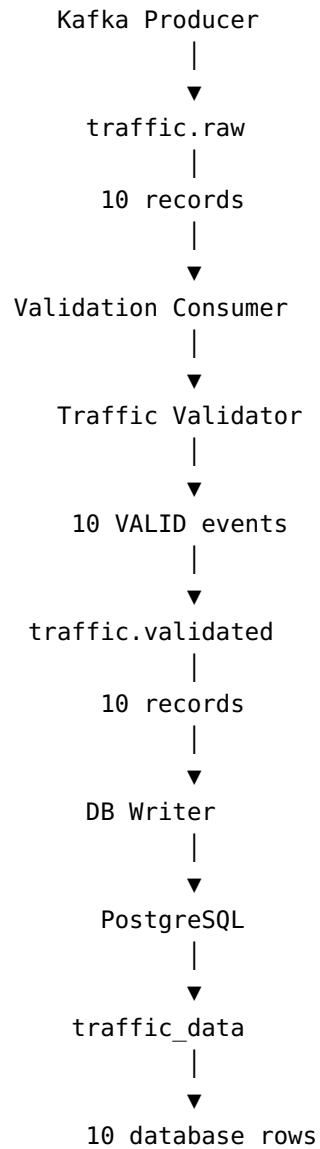
XML records	= 10
Kafka raw records	= 10
Validated records	= 10
PostgreSQL records	= 10

This provides an end-to-end consistency check.

44. End-to-End Test Summary

The complete test was:





45. Test Evidence

Test 1 — Python Environment

Result:

```
psycopg OK: 3.3.5  
kafka-python OK
```

Status: PASSED

Test 2 — Kafka Connection

Result:

```
Broker version identified as 4.0  
Connected
```

Status: PASSED

Test 3 — PostgreSQL Connection

Result:

```
Connected to PostgreSQL successfully
```

Status: PASSED

Test 4 — XML Ingestion

Command:

```
python -m scripts.produce_traffic --limit 10
```

Result:

```
Input events: 10  
Produced events: 10  
Failed events: 0
```

Status: PASSED

Test 5 — Raw Kafka Topic

Result:

```
traffic.raw:0:3  
traffic.raw:1:2  
traffic.raw:2:5
```

Total:

10

Status: PASSED

Test 6 — Validation Consumer

Result:

```
Processed valid events : 10  
Published validated    : 10
```

Status: PASSED

Test 7 — Validated Kafka Topic

Result:

10 validated messages successfully written

Status: PASSED

Test 8 — PostgreSQL Writer

Result:

```
Inserted batch of 10 records into PostgreSQL  
Kafka offsets committed successfully
```

Status: PASSED

Test 9 — Final Database Count

Command:

```
docker exec -it predictive-sdn-postgres
psql -U sdn_user -d sdn_traffic
-c "SELECT COUNT(*) FROM traffic_data;"
```

Result:

10

Status: PASSED

46. Overall Validation Result

The current pipeline has successfully passed an end-to-end integration test.

Stage	Input	Output	Result
XML Dataset	10	10	PASS
Kafka Producer	10	10	PASS
traffic.raw	10	10	PASS
Validation Consumer	10	10 valid	PASS
traffic.validated	10	10	PASS
PostgreSQL Writer	10	10	PASS
PostgreSQL	10	10 rows	PASS

Final consistency:

10 → 10 → 10 → 10

No records were lost during this test.

47. Important Kafka Consumer-Group Observation

During development, more than one DB writer process was accidentally started using the same consumer group:

```
postgres-db-writer
```

Kafka responded by rebalancing partitions.

For example, one process could receive:

```
partition 0  
partition 1
```

while another process receives:

```
partition 2
```

This is normal Kafka consumer-group behavior.

It is not an error.

However, during controlled testing, only one DB writer process should be running unless multiple workers are intentionally being used for parallel processing.

48. Kafka Consumer Group Concept

Kafka distributes partitions among consumers belonging to the same group.

For example:

```
Topic: traffic.validated  
  
Partition 0  
Partition 1  
Partition 2
```

One consumer:

Consumer A

└─ P0

└─ P1

└─ P2

Two consumers:

Consumer A

└─ P0

└─ P1

Consumer B

└─ P2

Three consumers:

Consumer A → P0

Consumer B → P1

Consumer C → P2

This architecture will become useful when processing millions of records.

49. Kafka Offset Management

A Kafka message is identified by:

Topic

Partition

Offset

For example:

traffic.raw

partition=2

offset=4

The DB schema preserves this information:

```
kafka_topic  
kafka_partition  
kafka_offset
```

This is useful for:

- debugging
- replay
- deduplication
- traceability
- auditing
- recovery

50. Data Lineage

The system maintains data lineage from source XML to PostgreSQL.

For example:

```
source_file  
  ↓  
source_folder  
  ↓  
event_id  
  ↓  
Kafka topic  
  ↓  
Kafka partition  
  ↓  
Kafka offset  
  ↓  
PostgreSQL row
```

This is important for a production-grade data engineering system.

If an incorrect database record is discovered, we should be able to trace it back to the Kafka message and eventually the original XML file.

51. Raw Payload Preservation

The PostgreSQL schema includes:

```
raw_payload JSONB
```

This preserves the original Kafka message.

This is useful because the database contains both:

```
Structured columns
```

and:

```
Original event payload
```

Structured columns are useful for analytics.

The raw payload is useful for:

- debugging
- auditing
- future parsing
- schema evolution
- investigation

52. Current Data Flow vs Original Desired Architecture

There is an important implementation detail.

The current producer performs validation before sending the event to `traffic.raw`:

```
DatasetLoader
  ↓
ValidationPipeline
  ↓
Kafka Producer
```

↓
traffic.raw

Then the validation consumer validates the event again:

traffic.raw
↓
TrafficKafkaConsumer
↓
Validation
↓
traffic.validated

Therefore, the current test contains validation at two levels.

The desired production architecture is cleaner:

XML
↓
Parser
↓
traffic.raw
↓
Validation Consumer
↓
traffic.validated
↓
PostgreSQL

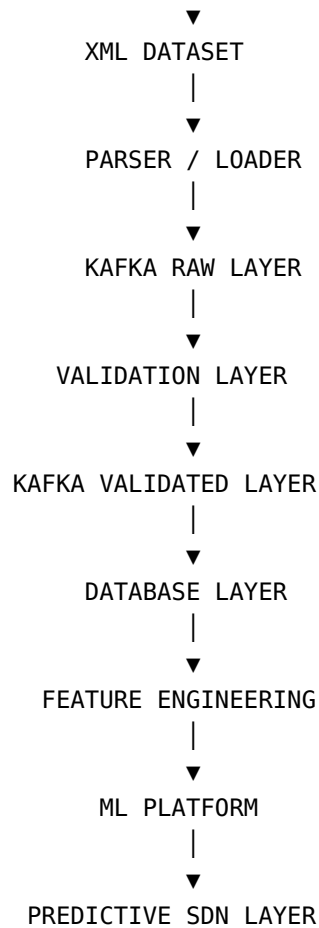
For the current integration test, the double validation is acceptable because it allowed the existing validation components to be tested safely.

Before processing the full 6-million-record dataset, the architecture should be adjusted so that validation has one clearly defined responsibility.

53. Current Architecture Maturity

The current system has reached the following stage:

DATA INGESTION LAYER
|



The first three major layers are now operational.

54. Known Warnings

During execution, the following warning appeared:

```
DeprecationWarning:  
value_serializer does not implement kafka.serializer.Serializer
```

and the DB writer showed:

```
DeprecationWarning:  
value_deserializer does not implement kafka.serializer.Deserializer
```

These are warnings rather than pipeline failures.

The Kafka producer and consumer still function correctly.

They should be cleaned up in a future refactoring so that the implementation follows the latest `kafka-python` serializer/deserializer interfaces.

55. Important Production Improvement — Offset Commit

The current validation consumer uses:

```
enable_auto_commit=False
```

but the existing consumer abstraction does not yet explicitly commit raw Kafka offsets after the validated message has been successfully published.

For a production-scale implementation, this needs to be improved.

The ideal flow is:

```
Consume raw event
  ↓
Validate
  ↓
Publish to traffic.validated
  ↓
Wait for Kafka acknowledgement
  ↓
Commit raw Kafka offset
```

This provides stronger processing guarantees.

56. Important Production Improvement — Batch Processing

The current test uses only:

10 records

The full dataset is much larger.

Processing millions of records one-by-one with excessive logging would be inefficient.

The production version should use:

```
Kafka batching
+
producer batching
+
database batch insertion
+
controlled logging
```

The DB writer already supports a batch size of:

5000

This should be benchmarked and tuned for the full workload.

57. Important Production Improvement — PostgreSQL Bulk Loading

The current writer uses batch insertion.

For millions of records, PostgreSQL `COPY` can potentially provide significantly better ingestion performance than repeated inserts.

Future optimization:

```
Kafka
↓
Batch
↓
COPY
```

↓
PostgreSQL

This should be evaluated after correctness has been established.

Correctness comes before optimization.

58. Important Production Improvement — Idempotency

The database already contains:

```
UNIQUE(topic, partition, offset)
```

This is an important foundation.

The database writer also uses:

```
ON CONFLICT DO NOTHING
```

Therefore, replayed Kafka messages should not create duplicate database rows based on the same Kafka coordinates.

This will become particularly important when:

- consumers restart
 - offsets are replayed
 - workers fail
 - deployments occur
 - large ingestion jobs are restarted
-

59. Important Production Improvement — Monitoring

The final system should monitor:

```
Kafka producer throughput
Kafka consumer lag
Validation success rate
Validation failure rate
DLQ count
PostgreSQL insertion rate
Database errors
Processing latency
ML prediction latency
```

Example metrics:

```
events_processed_total
events_validated_total
events_rejected_total
events_dlq_total
database_rows_inserted_total
kafka_consumer_lag
processing_latency_ms
```

60. Testing Strategy

Testing should occur progressively.

Level 1 — Unit Tests

Test:

```
XML parser
TrafficEvent
TrafficValidator
DuplicateDetector
Producer serialization
DB transformation
```

Level 2 — Component Tests

Test:

```
XML → TrafficEvent
TrafficEvent → Kafka
Kafka → Consumer
Consumer → Validator
Validated Event → Kafka
Kafka → PostgreSQL
```

Level 3 — Integration Test

The current 10-record test is an integration test.

```
XML
↓
Kafka
↓
Validation
↓
Kafka
↓
PostgreSQL
```

Result:

```
10 records successfully processed
```

Level 4 — Scale Test

Next:

```
100
1,000
10,000
100,000
1,000,000+
```

records.

Measure:

- throughput
 - CPU
 - memory
 - Kafka lag
 - PostgreSQL performance
 - end-to-end latency
-

61. Recommended Testing Progression

Do not immediately run all 6 million records.

Use:

```
10 records
↓
100 records
↓
1,000 records
↓
10,000 records
↓
100,000 records
↓
1 million
↓
Full dataset
```

At every stage record:

```
Input
Output
Failed
Throughput
Duration
Database count
```

62. Reproducibility

A successful test should always be reproducible.

The primary commands are:

Activate environment

```
source .venv/bin/activate
```

Start infrastructure

```
docker compose -f docker/kafka/docker-compose.yml up -d
```

Check containers

```
docker ps
```

Produce test data

```
python -m scripts.produce_traffic --limit 10
```

Start validation

```
python -m src.pipelines.validation_consumer
```

Start DB writer

```
python src/database/db_writer.py
```

Check database

```
docker exec -it predictive-sdn-postgres  
psql -U sdn_user -d sdn_traffic  
-c "SELECT COUNT(*) FROM traffic_data;"
```

63. Recommended Order for Running the System

For a clean test:

Step 1

Start Docker infrastructure:

```
docker compose -f docker/kafka/docker-compose.yml up -d
```

Step 2

Verify Kafka and PostgreSQL:

```
docker ps
```

Step 3

Start PostgreSQL DB writer:

```
python src/database/db_writer.py
```

Step 4

Start validation consumer:

```
python -m src.pipelines.validation_consumer
```

Step 5

Inject XML data:

```
python -m scripts.produce_traffic --limit 10
```

Step 6

Verify PostgreSQL:

```
docker exec -it predictive-sdn-postgres
psql -U sdn_user -d sdn_traffic
-c "SELECT COUNT(*) FROM traffic_data;"
```

Expected result for the first clean test:

```
10
```

64. Troubleshooting Lessons Learned

Problem 1 — `ModuleNotFoundError: No module named 'src'`

Command:

```
python scripts/produce_traffic.py
```

failed.

Solution:

```
python -m scripts.produce_traffic --limit 10
```

Reason:

Python needs to execute the script as part of the project package structure.

Problem 2 — PEP 668 externally managed environment

Attempting:

```
pip install psycopg
```

outside the virtual environment caused an externally-managed-environment error.

Solution:


```
source .venv/bin/activate
```

Then install dependencies inside `.venv`.

This avoids:

```
--break-system-packages
```

which should not be necessary for this project.

Problem 3 — Kafka `GetOffsetShell`

Older Kafka commands commonly use:

```
kafka-run-class.sh kafka.tools.GetOffsetShell
```

Kafka 4.0 provides:

```
/opt/kafka/bin/kafka-get-offsets.sh
```

Therefore the project uses the newer command.

Problem 4 — Multiple DB Writers

Starting multiple DB writers with the same group:

```
postgres-db-writer
```

causes Kafka partition rebalancing.

This is expected Kafka behavior.

During testing, run one DB writer unless parallel processing is intentional.

Problem 5 — PostgreSQL prompt confusion

SQL commands must be executed inside PostgreSQL or with:

```
psql -c
```

They should not be entered directly into Bash.

If PostgreSQL displays:

```
sdn_traffic-#
```

instead of:

```
sdn_traffic=#
```

it generally means PostgreSQL is waiting for completion of an unfinished SQL statement.

Press:

```
Ctrl+C
```

to cancel it.

65. Current Achievement

The project has now successfully demonstrated the following:

Historical XML network traffic can be loaded, transformed into normalized traffic events, injected into Apache Kafka, consumed and validated, published into a validated Kafka topic, and finally persisted into PostgreSQL with Kafka metadata and transactional offset handling.

This is the foundation required before moving into the ML and predictive load-balancing components.

66. Current End-to-End Proof

The most important proof obtained so far is:

```
SOURCE
10 XML events

      ↓

KAFKA RAW
10 events

      ↓

VALIDATION
10 valid events

      ↓

KAFKA VALIDATED
10 events

      ↓

POSTGRESQL
10 rows
```

Therefore:

```
SOURCE COUNT      = 10
RAW COUNT          = 10
VALIDATED COUNT    = 10
DATABASE COUNT     = 10
```

Result:

```
DATA CONSISTENCY = 100%
```

for the current 10-record integration test.

67. What Has NOT Yet Been Implemented

The following stages are still pending:

```
PostgreSQL
  ↓
Feature Engineering
  ↓
Feature Store / Feature Tables
  ↓
Training Dataset
  ↓
ML Model
  ↓
Model Evaluation
  ↓
MLflow Experiment Tracking
  ↓
Model Registry
  ↓
Inference Service
  ↓
Real-time Prediction
  ↓
SDN Controller
  ↓
Dynamic Load Balancing
```

These should be developed only after the ingestion and validation layer is stable.

68. Next Development Phase

The next major phase should be:

Phase 2 — Data Engineering and Feature Engineering

The next pipeline will be:

```
PostgreSQL
  ↓
Data Quality Checks
  ↓
```

```
Feature Engineering
  ↓
Training Dataset
  ↓
Train / Validation / Test Split
  ↓
Feature Store / Feature Tables
```

Potential features include:

```
traffic_mbps
rolling_mean_traffic
rolling_std_traffic
traffic_growth_rate
source_node
destination_node
demand_id
time_of_day
day_of_week
historical_load
node_utilization
link_utilization
```

Depending on the available source data, additional network features can be derived.

69. ML Stage

The eventual ML architecture will look like:

```
Validated Network Traffic
  ↓
Feature Engineering
  ↓
Training Dataset
  ↓
ML Model
  ↓
Traffic Prediction
  ↓
Congestion Prediction
  ↓
Load-Balancing Decision
```

Candidate models can include:

```
Linear Regression
Random Forest
XGBoost
LightGBM
LSTM
GRU
Temporal models
Transformer-based time-series models
```

The final model should be selected based on:

- prediction accuracy
- latency
- scalability
- interpretability
- resource requirements

70. MLOps Stage

MLflow will eventually be integrated for:

```
Experiment tracking
Model metrics
Model parameters
Model artifacts
Model registry
Model versioning
```

Architecture:

```
Training Pipeline
  ↓
MLflow
  ↓
Experiment
  ↓
Best Model
  ↓
Model Registry
```

↓
Production Model

71. Dynamic Load Balancing Stage

The final objective is to connect prediction to SDN.

Conceptually:

```
Network Traffic
  ↓
Kafka
  ↓
Feature Engineering
  ↓
ML Model
  ↓
Predicted Traffic
  ↓
Congestion Risk
  ↓
SDN Decision Engine
  ↓
Select Optimal Path
  ↓
Update Flow Rules
```

For example:

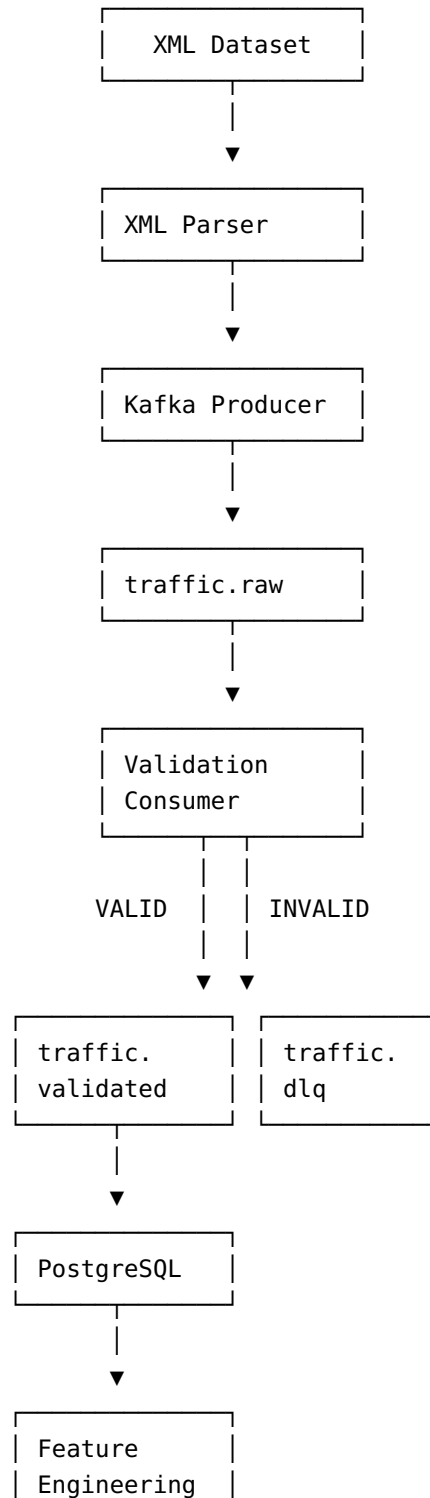
```
Path A → predicted congestion: HIGH
Path B → predicted congestion: LOW
Path C → predicted congestion: MEDIUM

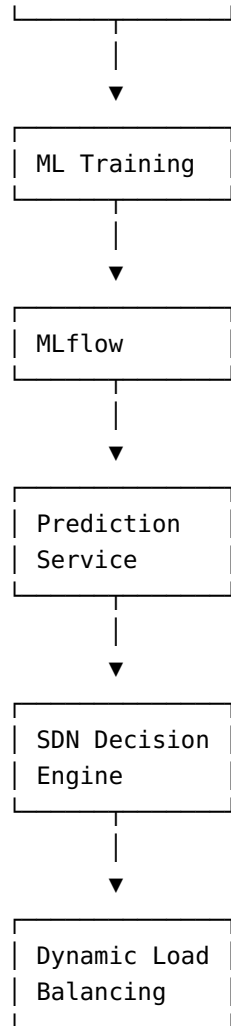
Decision:
Route traffic through Path B
```

This converts the project from a predictive analytics system into a predictive network control system.

72. Final Architecture Goal

The final architecture is expected to become:





73. Conclusion

The first major milestone of the **Predictive SDN Dynamic Load Balancer** has been successfully completed.

The project now has a working streaming data foundation:

```
XML
↓
Parser
↓
Kafka
↓
Validation
↓
```

```
Kafka
↓
PostgreSQL
```

The end-to-end test processed **10 records successfully**, with:

```
10 input records
10 Kafka raw records
10 validated records
10 PostgreSQL rows
0 failed records
```

The successful database verification:

```
count
-----
10
```

provides concrete evidence that the current ingestion and validation pipeline is functioning correctly.

The next focus should therefore be **scaling this pipeline safely from 10 records to larger batches**, followed by **feature engineering and ML dataset construction**.

Appendix A — Important Commands

Activate virtual environment

```
source .venv/bin/activate
```

Start Docker

```
docker compose -f docker/kafka/docker-compose.yml up -d
```

Check Docker

```
docker ps
```

Produce 10 test records

```
python -m scripts.produce_traffic --limit 10
```

Run validation consumer

```
python -m src.pipelines.validation_consumer
```

Run DB writer

```
python src/database/db_writer.py
```

Check raw Kafka offsets

```
docker exec -it predictive-sdn-kafka  
/opt/kafka/bin/kafka-get-offsets.sh  
--bootstrap-server localhost:9092  
--topic traffic.raw
```

Check validated Kafka offsets

```
docker exec -it predictive-sdn-kafka  
/opt/kafka/bin/kafka-get-offsets.sh  
--bootstrap-server localhost:9092  
--topic traffic.validated
```

Check PostgreSQL row count

```
docker exec -it predictive-sdn-postgres  
psql -U sdn_user -d sdn_traffic  
-c "SELECT COUNT(*) FROM traffic_data;"
```

Check database records

```
docker exec -it predictive-sdn-postgres  
psql -U sdn_user -d sdn_traffic
```

```
-c "SELECT id, event_id, source_node, destination_node, throughput_mbps,  
kafka_topic, kafka_partition, kafka_offset FROM traffic_data LIMIT 10;"
```

Appendix B — Current Milestone

PREDICTIVE SDN LOAD BALANCER	
XML Dataset	✓
XML Parser	✓
Dataset Loader	✓
Traffic Event Model	✓
Kafka Raw Topic	✓
Kafka Producer	✓
Kafka Consumer	✓
Traffic Validation	✓
Duplicate Detection	✓
DLQ	✓
Kafka Validated Topic	✓
PostgreSQL	✓
Kafka → PostgreSQL	✓
10-record E2E test	✓
Feature Engineering	→ NEXT
ML Training	→ NEXT
MLflow	→ NEXT
Prediction Service	→ NEXT
SDN Controller	→ FUTURE
Dynamic Load Balancing	→ FUTURE

Current milestone: DATA INGESTION + VALIDATION + DATABASE INTEGRATION — SUCCESSFULLY COMPLETED.